Practical 4: Classification Algorithms## Practical Information| Attribute | Details ||---------|---------|| **Practical Number** | 4 || **Title** | Classification Algorithms || **Duration** | 4 Hours || **Course Outcomes (COs)** | CO2, CO3, CO4 || **Description** | Implement and compare various classification algorithms on different datasets. |## Key Topics Covered- Logistic Regression- Decision Trees- Random Forest- SVM- Naive Bayes- Model evaluation---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 4: Classification Algorithms
# Write your implementation here
# Practical 4: Classification Algorithms
# Write your implementation here
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris  # Using iris dataset as an example
from sklearn.ensemble import RandomForestClassifier  # Example classifier

# Load a sample dataset
data = load_iris()
X = data.data
y = data.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train a model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Generate predictions
y_pred = model.predict(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Print results
print("📊 Evaluation Metrics\n")

print(f"Accuracy : {accuracy:.2f} ({accuracy*100:.2f}%)")
print(f"Precision: {precision:.2f} ({precision*100:.2f}%)")
print(f"Recall   : {recall:.2f} ({recall*100:.2f}%)")
print(f"F1-Score : {f1:.2f} ({f1*100:.2f}%)")

# Confusion Matrix
print("\n🔢 Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
📊 Evaluation Metrics

Accuracy : 1.00 (100.00%)
Precision: 1.00 (100.00%)
Recall   : 1.00 (100.00%)
F1-Score : 1.00 (100.00%)

🔢 Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
```

```
 [ 0  0 13]]
```

Assistant

The error occurs because you're trying to use variables `y_test` and `y_pred` that haven't been defined yet. The code assumes these variables already exist, but they don't.

Would you like me to provide the corrected code?

```
# Practical 4: Classification Algorithms
# Logistic Regression Implementation

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the model
model = LogisticRegression(max_iter=200, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Output
print("📊 Evaluation Metrics\n")

print(f"Accuracy : {accuracy:.2f} ({accuracy*100:.2f}%)")
print(f"Precision: {precision:.2f} ({precision*100:.2f}%)")
print(f"Recall   : {recall:.2f} ({recall*100:.2f}%)")
print(f"F1-Score : {f1:.2f} ({f1*100:.2f}%)")

# Confusion Matrix
print("\n🔢 Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
📊 Evaluation Metrics

Accuracy : 1.00 (100.00%)
Precision: 1.00 (100.00%)
Recall   : 1.00 (100.00%)
F1-Score : 1.00 (100.00%)

🔢 Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
# Practical 4: Classification Algorithms
# K-Nearest Neighbors (KNN) Implementation

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
data = load_iris()
```

```
X = data.data
y = data.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train the model
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Output
print("📊 Evaluation Metrics\n")

print(f"Accuracy : {accuracy:.2f} ({accuracy*100:.2f}%)")
print(f"Precision: {precision:.2f} ({precision*100:.2f}%)")
print(f"Recall   : {recall:.2f} ({recall*100:.2f}%)")
print(f"F1-Score : {f1:.2f} ({f1*100:.2f}%)")

# Confusion Matrix
print("\n🔢 Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
📊 Evaluation Metrics

Accuracy : 1.00 (100.00%)
Precision: 1.00 (100.00%)
Recall   : 1.00 (100.00%)
F1-Score : 1.00 (100.00%)

🔢 Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```